

joyerIA — Non Functional Requirements Document (NFR)

1. INFORMACIÓN GENERAL

Proyecto

joyerIA

Documento

Non Functional Requirements Document (NFR)

Objetivo

Definir los requerimientos no funcionales del MVP de joyerIA.

Este documento establece los estándares de calidad, rendimiento, seguridad, escalabilidad y experiencia técnica que el sistema debe cumplir.

2. ALCANCE

Este documento cubre:

- Performance
 - Escalabilidad
 - Seguridad
 - Disponibilidad
 - UX
 - Infraestructura
 - Compatibilidad
 - Observabilidad
 - Mantenibilidad
 - Calidad técnica
-

3. PRINCIPIOS TÉCNICOS DEL MVP

Objetivos principales

La arquitectura del MVP debe priorizar:

- Simplicidad inteligente.
 - Desarrollo rápido.
 - Escalabilidad razonable.
 - Bajo costo operativo.
 - Excelente experiencia móvil.
 - Modularidad.
 - Seguridad.
-

Filosofía técnica

El MVP NO debe construirse como una arquitectura enterprise compleja.

La prioridad será:

- Validar producto.
 - Iterar rápido.
 - Mantener flexibilidad.
 - Reducir complejidad operativa.
-

4. PERFORMANCE

RNF-PERF-001 — Tiempo de carga inicial

La aplicación debe cargar en menos de 3 segundos en conexiones móviles promedio.

RNF-PERF-002 — Feed fluido

El scroll vertical del feed debe mantenerse fluido sin bloqueos perceptibles.

RNF-PERF-003 — Reproducción de videos

Los videos deben iniciar reproducción en menos de 2 segundos.

RNF-PERF-004 — Lazy loading

El sistema debe implementar lazy loading para contenido multimedia.

RNF-PERF-005 — Optimización multimedia

Las imágenes y videos deben comprimirse automáticamente.

RNF-PERF-006 — Caché local

La aplicación debe utilizar caché local para mejorar experiencia offline parcial.

RNF-PERF-007 — Consumo eficiente de red

La aplicación debe minimizar consumo innecesario de ancho de banda.

5. ESCALABILIDAD

RNF-SCALE-001 — Arquitectura modular

La arquitectura debe permitir agregar nuevos módulos sin afectar el core principal.

RNF-SCALE-002 — Escalabilidad horizontal razonable

El backend debe soportar crecimiento progresivo de usuarios.

RNF-SCALE-003 — Separación de servicios IA

Los servicios de IA deben mantenerse desacoplados del core principal.

RNF-SCALE-004 — Escalabilidad multimedia

El sistema debe utilizar almacenamiento optimizado para multimedia.

RNF-SCALE-005 — Evolución futura

La arquitectura debe permitir migrar posteriormente a microservicios si el producto crece significativamente.

6. SEGURIDAD

RNF-SEC-001 — Comunicación segura

Toda comunicación debe utilizar HTTPS/TLS.

RNF-SEC-002 — Gestión segura de autenticación

La autenticación debe utilizar tokens seguros y expiración controlada.

RNF-SEC-003 — Protección de contraseñas

Las contraseñas nunca deben almacenarse en texto plano.

RNF-SEC-004 — Roles y permisos

El sistema debe implementar control de acceso basado en roles.

RNF-SEC-005 — Validación de archivos

El sistema debe validar archivos antes de almacenarlos.

RNF-SEC-006 — Protección contra abuso

El sistema debe implementar límites básicos contra spam y abuso.

RNF-SEC-007 — Protección de datos sensibles

La información sensible debe almacenarse de forma segura.

RNF-SEC-008 — Prevención de acceso no autorizado

Las APIs privadas deben requerir autenticación válida.

7. DISPONIBILIDAD

RNF-AVAIL-001 — Disponibilidad objetivo

El sistema debe buscar una disponibilidad mínima del 99%.

RNF-AVAIL-002 — Manejo de errores

La aplicación debe manejar errores sin cerrar inesperadamente.

RNF-AVAIL-003 — Recuperación ante fallos

El sistema debe permitir recuperación rápida ante errores críticos.

RNF-AVAIL-004 — Persistencia de datos

La información de usuarios y publicaciones no debe perderse ante fallos.

8. EXPERIENCIA DE USUARIO

RNF-UX-001 — Diseño mobile-first

Toda la experiencia debe diseñarse prioritariamente para dispositivos móviles.

RNF-UX-002 — UI premium

La interfaz debe transmitir sensación:

- Premium.
- Elegante.
- Moderna.
- Futurista.

RNF-UX-003 — Navegación intuitiva

La navegación debe minimizar fricción y cantidad de pasos.

RNF-UX-004 — Animaciones fluidas

Las animaciones deben sentirse suaves y naturales.

RNF-UX-005 — Estados visuales claros

La aplicación debe mostrar claramente:

- Loading.
 - Error.
 - Success.
 - Empty states.
-

RNF-UX-006 — Accesibilidad básica

La interfaz debe mantener tamaños y contrastes legibles.

9. COMPATIBILIDAD

RNF-COMP-001 — Soporte Android

La aplicación debe funcionar correctamente en Android moderno.

RNF-COMP-002 — Soporte iOS

La aplicación debe funcionar correctamente en iOS moderno.

RNF-COMP-003 — Responsive layouts

La interfaz debe adaptarse a diferentes tamaños de pantalla.

10. MANTENIBILIDAD

RNF-MAIN-001 — Arquitectura modular

El código debe organizarse por módulos y responsabilidades.

RNF-MAIN-002 — Separación de capas

Frontend, backend y servicios externos deben mantenerse desacoplados.

RNF-MAIN-003 — Código reutilizable

Los componentes UI y lógica deben ser reutilizables.

RNF-MAIN-004 — Convenciones de código

El proyecto debe utilizar convenciones consistentes.

RNF-MAIN-005 — Tipado fuerte

El sistema debe utilizar tipado fuerte cuando sea posible.

RNF-MAIN-006 — Escalabilidad de código

La estructura debe permitir crecimiento del proyecto sin degradar mantenibilidad.

11. OBSERVABILIDAD

RNF-OBS-001 — Logging

El sistema debe registrar errores importantes.

RNF-OBS-002 — Monitoreo

La plataforma debe permitir monitoreo básico de errores.

RNF-OBS-003 — Analytics

El sistema debe registrar métricas de uso principales.

12. BASE DE DATOS

RNF-DB-001 — Integridad de datos

La base de datos debe mantener integridad relacional.

RNF-DB-002 — Escalabilidad relacional

La estructura debe soportar crecimiento de entidades y relaciones.

RNF-DB-003 — Índices

Las consultas frecuentes deben optimizarse mediante índices.

RNF-DB-004 — Soft delete

El sistema debe permitir desactivación lógica cuando sea necesario.

RNF-DB-005 — Backups

La base de datos debe contar con backups automáticos.

13. STORAGE Y MULTIMEDIA

RNF-STORAGE-001 — Almacenamiento multimedia

El sistema debe soportar almacenamiento de:

- Imágenes.
 - Videos.
 - Archivos.
-

RNF-STORAGE-002 — Optimización multimedia

Los archivos deben optimizarse automáticamente.

RNF-STORAGE-003 — CDN

El sistema debe utilizar CDN para contenido multimedia.

RNF-STORAGE-004 — Límite de archivos

La plataforma debe controlar tamaños máximos de archivos.

14. IA Y SERVICIOS EXTERNOS

RNF-AI-001 — Desacoplamiento IA

Los servicios IA deben funcionar como capa independiente.

RNF-AI-002 — Manejo de errores IA

La aplicación debe manejar errores de APIs IA sin afectar estabilidad general.

RNF-AI-003 — Control de costos

La generación IA debe tener límites configurables.

RNF-AI-004 — Procesamiento asíncrono

Las operaciones IA deben ejecutarse de forma asíncrona cuando sea necesario.

15. MODERACIÓN Y CONFIANZA

RNF-MOD-001 — Sistema de reportes

La plataforma debe permitir reportar contenido inapropiado.

RNF-MOD-002 — Historial administrativo

Las acciones administrativas deben registrarse.

RNF-MOD-003 — Verificación de joyerías

La plataforma debe mantener procesos de verificación manual.

16. DEVOPS E INFRAESTRUCTURA

RNF-DEVOPS-001 — Deploy automatizado

El proyecto debe permitir despliegues simples y repetibles.

RNF-DEVOPS-002 — Entornos separados

Debe existir separación entre:

- Desarrollo.
 - Testing.
 - Producción.
-

RNF-DEVOPS-003 — Variables de entorno

Las credenciales deben almacenarse mediante variables de entorno.

RNF-DEVOPS-004 — Gestión de secretos

Las llaves sensibles no deben almacenarse en repositorios.

17. COSTOS Y OPTIMIZACIÓN

RNF-COST-001 — Infraestructura optimizada

La arquitectura MVP debe minimizar costos iniciales.

RNF-COST-002 — Optimización IA

La generación IA debe limitarse para evitar costos excesivos.

RNF-COST-003 — Optimización multimedia

El almacenamiento multimedia debe utilizar compresión.

18. MÉTRICAS TÉCNICAS IMPORTANTES

Métricas críticas

- Tiempo de carga.
 - Tiempo de reproducción.
 - Errores frontend.
 - Errores backend.
 - Tiempo de respuesta APIs.
 - Uso de IA.
 - Consumo multimedia.
-

19. TECNOLOGÍAS PROPUESTAS

Frontend

Flutter.

Backend

Supabase.

Base de datos

PostgreSQL.

Realtime

Supabase Realtime.

Push notifications

Firebase Cloud Messaging.

IA

OpenAI APIs.

Storage

Supabase Storage + CDN.

20. CONCLUSIÓN

Este documento establece los estándares técnicos mínimos necesarios para garantizar que el MVP de joyería sea:

- Escalable.
- Seguro.
- Mantenible.
- Fluido.
- Preparado para evolucionar.

La prioridad principal será construir una arquitectura moderna, simple y flexible que permita validar rápidamente el producto sin generar complejidad innecesaria.